
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 10

Estruturas, Uniões, Manipulações de Bits e Enumerações
em C

Exercícios de Autorrevisão (10.1–10.4)

Resolvidos passo a passo, abordagem incremental
Capítulos 1 a 10

Construindo sobre os Capítulos Anteriores

No Capítulo 10, ampliamos drasticamente nossa capacidade de modelar dados em C. Construindo sobre os fundamentos dos Caps. 1–9, incorporamos quatro famílias de recursos poderosos:

- **Estruturas** (`struct`) – agrupam variáveis *heterogêneas* (tipos diferentes) sob um único nome, criando “registros”
- **Uniões** (`union`) – estruturas onde os membros *compartilham* o mesmo espaço de memória
- **Operadores sobre bits** – `&`, `|`, `^`, `~`, `«`, `»` para manipulação em nível de bits
- **Campos de bit** – membros de estrutura com largura específica em bits
- `typedef` – cria sinônimos para tipos existentes
- **Enumerações** (`enum`) – conjuntos nomeados de constantes inteiras

Boa prática de programação

As `struct` são a ponte para conceitos de programação orientada a objetos. Antes de `struct`, manipulávamos apenas tipos primitivos isolados ou arrays homogêneos. Agora podemos criar *tipos próprios* – a base do design moderno de software.

Contents

1	Exercício 10.1 – Preenchimento de Lacunas (14 itens)	2
2	Exercício 10.2 – Verdadeiro ou Falso	3
3	Exercício 10.3 – Código com <code>struct</code> peça	5
4	Exercício 10.4 – Identificar e Corrigir Erros	6

1. Exercício 10.1 – Preenchimento de Lacunas (14 itens)

Resposta detalhada

- a) Uma **estrutura** (**struct**) é uma coleção de variáveis relacionadas agrupadas sob um único nome.
- b) Uma **união** (**union**) é uma coleção de variáveis sob um único nome que *compartilham o mesmo espaço de armazenamento*.
- c) Os bits no resultado de uma expressão com o operador **AND sobre bits** (**&**) são 1 se os bits correspondentes em *ambos* os operandos são 1.
- d) Variáveis declaradas em uma definição de estrutura são chamadas de **membros**.
- e) Em expressões com **OR inclusivo sobre bits** (**|**), os bits são 1 se *pelo menos um* dos bits correspondentes for 1.
- f) A palavra-chave **struct** introduz uma declaração de estrutura.
- g) A palavra-chave **typedef** cria um sinônimo para um tipo previamente definido.
- h) Em expressões com **OR exclusivo (XOR) sobre bits** (**^**), os bits são 1 se *exatamente um* dos bits correspondentes for 1.
- i) O AND sobre bits é normalmente usado para **maskar** bits (selecionar certos e zerar outros).
- j) A palavra-chave **union** introduz uma definição de união.
- k) O nome da estrutura é chamado de **nome de tag** (*tag name*).
- l) Um membro pode ser acessado com o operador **ponto** (**.**) ou **seta** (**->**), este último quando temos um ponteiro para a estrutura.
- m) **«** (deslocamento à esquerda) e **»** (deslocamento à direita) deslocam os bits de um valor.
- n) Uma **enumeração** (**enum**) é um conjunto de inteiros representados por identificadores.

2. Exercício 10.2 – Verdadeiro ou Falso

Item (a) – struct só com um tipo

FALSO

Esta é justamente a grande vantagem das estruturas sobre arrays: **struct** pode conter **muitos tipos diferentes**. Esse é o ponto-chave que diferencia **struct** de array (que é homogêneo).

```
1 struct aluno {
2     char nome[ 50 ];    /* char[] */
3     int  matricula;     /* int    */
4     double media;       /* double */
5 };
```

Item (b) – Uniões podem ser comparadas com ==

FALSO

Uniões **não** podem ser comparadas diretamente porque podem ter *bytes indefinidos* (padding) com valores diferentes mesmo quando o conteúdo lógico é o mesmo. A comparação byte-a-byte poderia falhar incorretamente. Para comparar uniões, compare membros individuais de forma explícita.

Item (c) – Nome da tag é opcional

VERDADEIRO

Uma estrutura pode ser declarada sem nome de tag (estrutura anônima), útil geralmente combinada com **typedef**:

```
1 typedef struct {          /* sem tag */
2     int x, y;
3 } Ponto;
```

Item (d) – Membros de structs diferentes devem ter nomes únicos

FALSO

Membros de *estruturas diferentes* podem ter nomes iguais sem conflito, porque o acesso é qualificado pelo nome da estrutura: **a.nome** vs **b.nome**. Apenas dentro da **mesma** estrutura os nomes devem ser únicos.

```
1 struct ponto2d { double x, y; };
2 struct ponto3d { double x, y, z; }; /* x e y aqui sao
   OK */
```

Item (e) – typedef define novos tipos

FALSO

`typedef` *não cria* novos tipos – ele apenas cria **sinônimos** (*aliases*) para tipos existentes. `Idade` e `int` são intercambiáveis se `typedef int Idade;`.

```
1 typedef int Idade;
2 Idade a = 30;      /* equivale a int a = 30; */
3 int b = a;         /* sem conversao -- mesmo tipo */
```

Item (f) – structs sempre passadas por referência

FALSO

Estruturas são passadas **por valor** por padrão em C – exatamente como qualquer outro tipo. Isso pode ser *caro* para estruturas grandes, pois *toda a estrutura é copiada*.

Para passar por referência, passe um ponteiro explicitamente:

```
1 void f( struct grande s );          /* por valor:
   COPIA tudo */
2 void f( struct grande *s );        /* por referencia
   */
```

Boa prática de programação

Para `struct` grandes, prefira passar por ponteiro. Use `const` quando não pretende modificar a estrutura – princípio do menor privilégio.

Item (g) – structs não podem ser comparadas com ==

VERDADEIRO

`structs` não podem ser comparadas diretamente com `==` ou `!=`, pelos mesmos motivos das uniões: bytes de *padding* (preenchimento para alinhamento) podem ter valores indefinidos.

Para comparar duas `struct`, compare membro por membro:

```
1 if ( a.x == b.x && a.y == b.y ) { ... }
```

3. Exercício 10.3 – Código com struct peça

Resposta detalhada

a) Definir struct peça:

```
1 struct peça {  
2     int    numPeca;  
3     char  nomePeca[ 26 ]; /* 25 chars + '\0' */  
4 };
```

b) typedef para criar Peça:

```
1 typedef struct peça Peça;
```

c) Declarações usando Peça:

```
1 Peça a;           /* variavel simples */  
2 Peça b[ 10 ];     /* array de 10 Pecas */  
3 Peça *ptr;        /* ponteiro para Peça */
```

Nota: com typedef podemos colocar tudo em uma linha: `Peça a, b[10], *ptr;`.

d) Ler número e nome para os membros de a:

```
1 scanf( "%d%25s", &a.numPeca, a.nomePeca );
```

Cuidado: `a.nomePeca` (sem `&`) porque é um array; o nome já é o endereço.

e) Atribuir a ao elemento 3 de b:

```
1 b[ 3 ] = a; /* C suporta atribuicao entre structs  
            */
```

f) Atribuir endereço de b a ptr:

```
1 ptr = b; /* nome do array = endereco do 1o  
          elemento */
```

g) Imprimir elemento 3 via ptr:

```
1 printf( "%d %s\n", ( ptr + 3 )->numPeca,  
2          ( ptr + 3 )->nomePeca );
```

Explicação: `ptr + 3` é aritmética de ponteiro (Cap. 7) – aponta para `b[3]`. O operador `->` é equivalente a `(*p).membro`, mas mais legível.

Boa prática de programação

A combinação `struct` + `typedef` + ponteiro é o tripé que sustenta todo o desenvolvimento orientado a tipos em C. Listas encadeadas (Cap. 12), árvores e sistemas grandes usam exatamente esse padrão.

4. Exercício 10.4 – Identificar e Corrigir Erros

Item (a) – Precedência de * e ->

Resposta detalhada

Código com erro:

```
1 printf( "%s\n", *cPtr->face );
```

Erro: O operador -> tem precedência *maior* que *. A expressão é avaliada como *(cPtr->face), o que tenta desreferenciar o char * (retornando um char), incompatível com %s que espera char *.

Correção: adicionar parênteses para forçar a ordem desejada:

```
1 printf( "%s\n", ( *cPtr ).face );
```

Mais idiomático seria simplesmente:

```
1 printf( "%s\n", cPtr->face ); /* equivalente */
```

Item (b) – Subscrito de array faltando

Resposta detalhada

Código com erro:

```
1 printf( "%s\n", copas.face ); /* tentava acessar  
    elemento 10 */
```

Erro: copas é um array (hearts[13]), não uma estrutura individual. Para acessar o décimo elemento (índice 10), use subscrito:

```
1 printf( "%s\n", copas[ 10 ].face );
```

(Notar que “elemento 10” tradicionalmente significa o de índice 9 ou 10 a depender de convenção; o livro usa 10.)

Item (c) – Inicialização de union

Resposta detalhada

Código com erro:

```
1 union valores {  
2     char w;  
3     float x;  
4     double y;  
5 };  
6 union valores v = { 1.27 };
```

Erro: Em uniões inicializadas com a sintaxe de chaves, a inicialização aplica-se *apenas ao primeiro membro*. Como o primeiro membro é char w, não dá

para inicializá-lo com 1.27 (um double).

Correção: reordenar a união ou usar inicializador designado (C99):

```
1  /* Opcao 1: reordenar */
2  union valores {
3      double y;
4      char w;
5      float x;
6  };
7  union valores v = { 1.27 };    /* OK: y = 1.27 */
8
9  /* Opcao 2: inicializador designado (C99) */
10 union valores v = { .y = 1.27 };
```

Item (d) – Falta ;

Resposta detalhada

Código com erro:

```
1  struct pessoa {
2      char nome[ 15 ];
3      char sobrenome[ 15 ];
4      int idade;
5  }
```

Erro: Falta o ; depois do }. Diferente do que acontece com funções (que não têm ; após }), declarações de struct (e union, enum) precisam de ; no final.

Correção:

```
1  struct pessoa {
2      char nome[ 15 ];
3      char sobrenome[ 15 ];
4      int idade;
5  };    /* <-- nao esqueca este ; */
```

Erro comum de programação

Esquecer o ; após } de struct/union/enum é um erro extremamente comum, e o compilador pode emitir mensagens confusas apontando para a próxima linha.

Item (e) – Falta struct

Resposta detalhada

Código com erro:

```
1  pessoa d;    /* assumindo definicao do item (d) ja
    corrigida */
```

Erro: Sem typedef, é necessário usar o nome completo `struct pessoa` para declarar variáveis.

Correção:

```
1  struct pessoa d;
```

Alternativa: usar typedef:

```
1  typedef struct pessoa Pessoa;
2  Pessoa d;    /* agora funciona */
```

Item (f) – Atribuição entre tipos diferentes

Resposta detalhada

Código com erro:

```
1  struct pessoa p;
2  struct card c;
3  p = c;
```

Erro: Em C, atribuição entre `struct` só funciona entre o **mesmo tipo**. `struct pessoa` e `struct card` são tipos distintos, mesmo que tivessem layout idêntico.

Não há correção direta: a operação não faz sentido semanticamente. Se realmente quiséssemos copiar campos compatíveis, faríamos manualmente:

```
1  strcpy( p.nome, c.algumCampo );
2  /* ... outros campos ... */
```

Boa prática de programação

C é **tipado nominalmente**: dois tipos `struct` são diferentes mesmo se tiverem o mesmo layout. Isso é uma forma de proteção contra erros – impede misturar conceitos por acidente.

Gabarito Rápido – Capítulo 10: Autorrevisão

Respostas Resumidas

10.1:

- a) estrutura b) união c) AND &
- d) membros e) OR inclusivo | f) `struct`
- g) `typedef` h) XOR ^ i) mascarar
- j) `union` k) nome de tag l) . ou ->
- m) « e » n) enumeração

10.2 – Verdadeiro/Falso:

- a) F (`struct` aceita múltiplos tipos)
- b) F (uniões têm bytes indefinidos)
- c) V (nome de tag é opcional)
- d) F (apenas dentro da mesma `struct`)
- e) F (`typedef` cria sinônimos, não tipos novos)
- f) F (`struct` é passada por valor)
- g) V (não comparáveis com ==)

10.3: `struct peca {...}; typedef ... Peca; Peca a, b[10], *ptr;;`
`scanf` com `&a.num` e `a.nome`; `b[3] = a; ptr = b; (ptr+3)->num.`

10.4 – Erros corrigidos:

- a) precedência * vs ->: use `(*cPtr).face` ou `cPtr->face`
- b) falta subscrito de array
- c) tipo do primeiro membro da união
- d) falta ; após } da `struct`
- e) falta palavra `struct` na declaração
- f) não se pode atribuir entre tipos `struct` diferentes